

Aprendiendo a jugar a Othello

Propuesta de trabajo para la asignatura Inteligencia Artificial

Profesor: Ignacio Pérez Hurtado de Mendoza

Grado en Ingeniería Informática - Ingeniería del Software
Curso 2025/2026

Introducción y objetivo

En este trabajo exploraremos el desarrollo de un agente inteligente para el clásico juego de **Othello** (también conocido como Reversi), utilizando herramientas modernas de **Inteligencia Artificial**. El objetivo es construir jugadores capaces de tomar decisiones estratégicas mediante la combinación de algoritmos de búsqueda y redes neuronales.

Se proponen dos variantes del proyecto:

- **Versión 1:** Implementaremos un agente basado en Monte Carlo Tree Search (MCTS) con el enfoque de **Upper Confidence Bound Tree (UCT)**. En esta versión, una red neuronal será utilizada para estimar el valor de las posiciones, guiando así la expansión y selección de nodos dentro del árbol de búsqueda.
- **Versión 2:** Se desarrollará un agente que utiliza el clásico algoritmo de **Minimax** con poda alfa-beta para explorar el espacio de decisiones. También en este caso, una red neuronal será utilizada, pero ahora para proporcionar una heurística de evaluación de las posiciones intermedias, reemplazando funciones de evaluación manuales.

La integración de MCTS y Deep Learning ya se utilizó en los famosos Alpha Go[4] y Alpha Zero[5], aunque la aproximación que haremos en este trabajo será mucho más sencilla y reducida. Por una parte, la complejidad del juego del Othello es mucho menor, y por otra, el enfoque algorítmico será también simplificado.

Lecturas recomendadas

Para poder entender el planteamiento y el alcance de este trabajo, se recomienda encarecidamente comenzar por un estudio y análisis de los algoritmos fundamentales que se van a utilizar y como se han aplicado a otros juegos, tales como el go o el ajedrez. En particular, habría que comenzar leyendo lo siguiente (y se recomienda el orden propuesto):

- **Búsqueda adversaria:** Para entender los conceptos de búsqueda adversaria, habría que leer el capítulo 5 del libro "Artificial Intelligence, a modern approach" [2]. Este libro se puede conseguir en la biblioteca de la Escuela. En ese capítulo se detallan no sólo los conceptos teóricos fundamentales, sino que se plantea el pseudocódigo del algoritmo minimax con poda alfa-beta, que deberá ser usado en este trabajo.
- **Métodos MTCS:** La lectura del artículo "A survey of Monte Carlo Tree Search Methods" [3] será necesaria para entender como MCTS se usa en búsqueda adversaria, de forma complementaria y, en muchas ocasiones, superando aproximaciones clásicas. En dicho artículo se incluye el pseudocódigo del algoritmo UCT, que deberá ser usado en este trabajo.
- **Artículos Alpha Go y Alpha Zero:** Para entender como se ha podido incorporar Deep Learning para mejorar la búsqueda adversaria y conseguir agentes tremendamente exitosos, se recomienda leer los artículos sobre Alpha Go [4] y Alpha Zero [5]

Versión a entregar en cada convocatoria

- **Versión a entregar en junio: UCT + Deep Learning**

Si se va a entregar el trabajo en la convocatoria de junio, se deberá incluir una implementación propia del algoritmo UCT (siguiendo el pseudocódigo en [3]) para el juego de Othello y una red neuronal entrenada para predecir la función DefaultPolicy(s).

- **Versión a entregar en julio: Minimax alfa-beta + Deep Learning**

En el caso de entregar el trabajo en la convocatoria de julio, se deberá incluir una implementación propia del algoritmo minimax alfa-beta (siguiendo el pseudocódigo en [2]) para el juego Othello y una red neuronal entrenada para predecir la función `game.utility(state,player)`. Se recomienda utilizar la versión del algoritmo con una profundidad limitada (cutting-off search).

- **Versión para entregar en tercera convocatoria (octubre-noviembre)**

En este caso, se podrá decidir una de las dos versiones anteriores.

Entrenamiento de la Red Neuronal

Para poder entrenar una red neuronal que ayude a evaluar posiciones en el juego, primero es necesario contar con un agente que ya pueda jugar de forma básica. Este agente debe estar implementado usando alguno de los algoritmos vistos: **MCTS con UCT** o **Minimax con poda Alpha-Beta** (según la variante del trabajo que corresponda).

El proceso de generación de datos para entrenar la red será el siguiente:

1. **Generar partidas automáticas:** Hacer que la máquina juegue contra sí misma utilizando el agente desarrollado. Durante cada partida, se deben registrar todos los estados intermedios del tablero.

2. **Guardar los estados:** Cada vez que un jugador realice una jugada, se debe guardar el estado resultante del tablero. Este estado puede representarse como una matriz de números:

- **0:** casilla vacía.
- **1:** casilla ocupada por una ficha blanca.
- **2:** casilla ocupada por una ficha negra.

(No es necesario guardar imágenes gráficas; basta con la representación matricial.)

3. **Etiquetar los estados:** A cada estado guardado se le debe asociar un valor de resultado:

- **+1** si el jugador activo en ese estado ganó la partida.
- **0** si hubo empate.
- **-1** si el jugador activo perdió.

4. **Diseñar y entrenar la red neuronal:** Usando los conocimientos adquiridos en la asignatura y herramientas como **TensorFlow**, se deberá diseñar una red neuronal que, a partir de un estado de juego, prediga un valor numérico en el rango $[-1, 1]$ indicando la probabilidad de victoria.

5. **Utilizar la red en el agente:** Una vez entrenada, la red neuronal sustituirá la función de evaluación tradicional:

- En **MCTS**, reemplazará a la función `DefaultPolicy`.
- En **Minimax**, reemplazará a la función `Utility` o heurística.

6. **(Opcional) Reentrenamiento iterativo:** Es posible realizar varias rondas de entrenamiento. Tras entrenar una primera versión de la red, se puede generar un nuevo conjunto de datos jugando partidas con el nuevo agente, y entrenar una red mejorada. Este proceso puede repetirse varias veces para perfeccionar el rendimiento.

Mínimos a cumplir

Independientemente de la convocatoria a entregar, cómo mínimo el funcionamiento del algoritmo de búsqueda (ya sea UCT o MiniMax alfa-beta) debe ser correcto. Tiene que ser posible jugar contra la máquina, el interfaz puede ser texto (mostrando en ASCII el estado del tablero y preguntando por la siguiente jugada al usuario) o gráfico (por ejemplo, usando PyGame). Pero si el núcleo de UCT o Minimax alfa-beta no es correcto, no se valorará el interfaz. Para estas versiones mínimas, no hace falta usar una red neuronal. En el caso de UCT, se sigue el pseudocódigo de la función `DefaultPolicy(s)` descrito en [3]. Por otra parte, en el caso de minimax alfa-beta, basta con desarrollar una función de heurística sobre el estado de un juego, por ejemplo, la diferencia entre fichas blancas y negras. En el caso de usar diferentes funciones heurísticas, se valorará la realización de alguna comparativa.

Documentos a entregar y descripción de la evaluación

La entrega del trabajo contendrá dos partes:

- Código fuente en Python
- Documentación

Adicionalmente, será obligatorio realizar una presentación y defensa ante el profesor, una vez hecha la entrega. Para ello, el profesor convocará a cada grupo de alumnos por email. La superación de esta presentación y defensa es requisito obligatorio.

Documentación

La documentación debe tener un mínimo de 6 páginas y un máximo de 12 páginas. Se valorará la utilización del sistema $\text{\LaTeX}$ [6] y se deberá seguir un esquema de artículo científico, de acuerdo al formato *IEEE Conference Proceedings* [7]. La documentación deberá contener las siguientes secciones:

- Resumen o *Abstract*
- Introducción
- Preliminares
- Implementación
- Pruebas y experimentación
- Conclusiones
- Bibliografía

Presentación y defensa

Como parte de la evaluación del trabajo se deberá realizar una presentación y defensa del mismo, para lo que se citarán a los alumnos de manera conveniente. En esa reunión se deberá llevar preparada y realizar una presentación (PDF, PowerPoint o similar) de 20 minutos en la que deberán participar activamente todos los miembros del grupo. Cada miembro debe hablar por igual. Esta presentación deberá seguir a grandes rasgos la misma estructura de la documentación entregada, haciendo especial mención a los resultados obtenidos y al análisis crítico de los mismos. En los siguientes 20-40 minutos, el profesor procederá a hacer preguntas a cada uno de los miembros del grupo, que podrán ser preguntas sobre la documentación, el código realizado o cuestiones técnicas, teóricas o algorítmicas. Las preguntas podrán estar dirigidas a los dos miembros del grupo o sólo a uno de ellos. Adicionalmente, el profesor podrá solicitar la realización sobre la marcha de pequeñas modificaciones en el código entregado y/o pruebas adicionales. Cada uno de los alumnos del grupo deberá demostrar un completo dominio sobre todas las partes del código y la documentación entregada, independientemente de la manera en la cual se hayan distribuido el trabajo. Para aprobar este trabajo es obligatorio aprobar la parte

de la presentación y defensa, siendo incluso posible que para un grupo de dos alumnos, uno de ellos supere la presentación y defensa y el otro no.

Uso de inteligencia artificial generativa

El uso de sistemas de inteligencia artificial generativa está permitido con las siguientes condiciones:

- Debe explicarse para qué se han utilizado esos sistemas, así como describir las entradas (prompts) proporcionadas a los mismos.
- En la defensa del trabajo ambos alumnos deben demostrar conocimiento y entendimiento de la totalidad del trabajo, lo que incluye las respuestas obtenidas de esos sistemas

Criterios de evaluación

Para la evaluación se tendrá en cuenta el siguiente criterio de valoración, considerando una nota máxima de 4 en total para el trabajo:

- **Código fuente y experimentación (hasta 1.5 puntos):** se valorará la claridad y buen estilo de programación, corrección, eficiencia y usabilidad de la implementación, así como calidad de los comentarios. En ningún caso se evaluará un trabajo con código copiado directamente de Internet o de otros compañeros.
- **Documentación (hasta 1.5 puntos):** se valorará la claridad de las explicaciones, el razonamiento de las decisiones, el análisis y presentación de resultados y el correcto uso del lenguaje. La elaboración de la memoria debe ser original, por lo que no se evaluará el trabajo si se detecta cualquier copia del contenido.
- **Presentación y defensa (hasta 1 punto):** se valorará la claridad de la presentación y la buena explicación de los contenidos del trabajo, así como las respuestas a las preguntas realizadas por el profesor. Para aprobar el trabajo es obligatorio sacar al menos 0.5 puntos en la presentación y defensa del mismo, pudiendo obtener una nota diferente cada uno de los integrantes de un grupo.

IMPORTANTE: Cualquier plagio, compartición de código o uso de material que no sea original y del que no se cite convenientemente la fuente, significará automáticamente la calificación de cero en la asignatura para todos los alumnos involucrados. Por tanto, a estos alumnos no se les conserva, ni para la actual ni para futuras convocatorias, ninguna nota que hubiesen obtenido hasta el momento. Todo ello sin perjuicio de las correspondientes medidas disciplinarias que se pudieran tomar.

Anexo: El juego Otelo

Otelo (también conocido como *Reversi*)¹ es un juego de estrategia para dos jugadores que se disputa sobre un tablero de **8x8** casillas.

Objetivo del Juego

El objetivo es tener la mayoría de fichas de tu color en el tablero al finalizar la partida.

Preparación Inicial

Al comienzo del juego, el tablero se dispone con cuatro fichas en el centro:

- Dos fichas blancas en posiciones opuestas.
- Dos fichas negras en las otras dos posiciones centrales.

El jugador con fichas negras mueve primero.

Reglas de Movimiento

- En su turno, un jugador debe colocar una ficha de su color sobre una casilla vacía de manera que, en línea recta (horizontal, vertical o diagonal), **una o más fichas del oponente queden atrapadas** entre la ficha que se coloca y otra ficha del propio jugador.
- Todas las fichas del oponente que queden atrapadas de esta manera **se voltean** y pasan a ser del color del jugador que hizo la jugada.
- Si un jugador no puede realizar un movimiento válido, debe pasar su turno.
- Si ambos jugadores no pueden realizar movimientos válidos, la partida finaliza.

Final del Juego

El juego termina cuando:

- Todas las casillas del tablero están ocupadas.
- Ninguno de los dos jugadores puede realizar un movimiento válido.

Condiciones de Victoria

Gana el jugador que tenga más fichas de su color en el tablero al final de la partida.

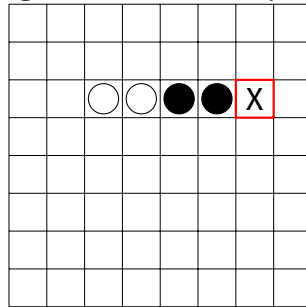
Notas Adicionales:

- Es obligatorio realizar un movimiento si es posible.

¹<https://es.wikipedia.org/wiki/Reversi>

- No está permitido colocar fichas de manera que no se capture ninguna ficha del oponente.

Ejemplo de jugada en Othello: captura horizontal



Colocando una ficha blanca en la casilla marcada (X), las fichas negras quedan atrapadas entre fichas blancas y son volteadas.

Bibliografía

- [1] Russell, Stuart and Norvig, Peter. Artificial Intelligence: A Modern Approach (2010). Chapter 22.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020, ch. 5, “Adversarial Search”.
- [3] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavenier, D. Perez, S. Samothrakis, and S. Colton, “A Survey of Monte Carlo Tree Search Methods,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012, doi: 10.1109/TCIAIG.2012.2186810.
- [4] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016, doi: 10.1038/nature16961.
- [5] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, “Mastering the game of Go, Chess, and Shogi by self-play with a general reinforcement learning algorithm,” *arXiv preprint*, arXiv:1712.01815, 2017. arXiv:1712.01815.
- [6] https://es.wikibooks.org/wiki/Manual_de_LaTeX
- [7] <https://www.ieee.org/conferences/publishing/templates.html>